
TECHNICAL RESEARCH & VERIFICATION REPORT

Technical Evaluation and Reproduction of F28 Script Matching Results

Lead Researcher	QAHTAN AL-LOUD
Status	Archived Reference Copy
Core Experiment	F28 Script Matching / CBigNum Serialization
F28 Exact Match Count	8,713

1. Executive Summary

This report documents the technical results of reproducing and executing the F28 analysis associated with early Bitcoin block data and byte-level serialization matching. Version 3 is maintained as the archived reference copy after refinement of the methodological boundaries and reproducibility description.

The experiment compares observed coinbase ScriptSig push[1] payloads with outputs generated by a historical CBigNum::getvch()-style serialization model. The reported F28 result is an exact byte-level matching count under the specified dataset, extraction rule, serialization implementation, and numerical range.

2. Methodological Structure

Component	Description
Observed data	Raw records produced by direct analysis of early block ScriptSig data.
CBigNum::getvch() model	Historical integer-to-byte serialization behavior implemented for the experiment.
F28 test	Programmatic search and exact byte-for-byte matching procedure.
Numerical result	8,713 exact matches under the defined experimental conditions.
Reproduction	Deterministic execution using a fixed dataset and defined extraction/model rules.
Interpretation and limits	Results are separated from miner identity, attribution, and unsupported external conclusions.

3. Observed Data

- **Data Source:** Early Bitcoin blockchain ScriptSig transaction records.
- **Extraction Rule:** Parse Script push operations and isolate the second pushed payload (push[1]).
- **Normalization:** Normalize observed payloads to lowercase hexadecimal representations and collect unique byte patterns.
- **Observed Population:** Exactly **9,993 unique push[1] payloads** in the analyzed dataset.

4. Computational Test & Serialization Model

A Python implementation of the legacy CBigNum::getvch()-style serialization behavior was used to generate deterministic byte representations for positive integer values.

- Convert the absolute integer value into a variable-length byte representation.
- Store the resulting bytes in little-endian order.
- Apply the legacy sign-bit rule used by the sign-magnitude representation.
- Use the resulting byte sequence as the exact hexadecimal comparison value.

Tested range: The simulation evaluated integers from **1 through 199,999 inclusive**, producing **199,999 simulated byte-serialization patterns**.

5. F28 Result

Each generated CBigNum serialization output was compared directly against the observed push[1] payload set. A match was recorded only upon strict, complete hexadecimal byte identity.

Parameter / Metric	Value
Observed Unique push[1] Payloads	9,993
Simulated CBigNum Values	199,999 (1–199,999 inclusive)
Exact Serialization Matches (F28)	8,713

Definitive F28 Result Statement

"F28 produced 8,713 byte-level serialization matches between observed coinbase push[1] payloads and the historical CBigNum getvch() serialization model over the tested ExtraNonce range of 1–199,999."

6. Earlier 78 Boundary Cases

Earlier investigative phases included a separate set of 78 boundary cases involving lower-bound integer representations and standard OP_PUSHDATA boundary conditions. These preliminary cases informed refinement of the extraction workflow but are treated as distinct from the definitive F28 serialization population and are not included in the 8,713 F28 count.

7. Reproduction Procedure

```
Input Dataset (ScriptSig records)
↓
Script push parsing & normalization
↓
push[1] extraction (9,993 unique patterns)
↓
CBigNum getvch() simulation (1-199,999)
↓
Exact hexadecimal byte comparison
↓
F28 output: 8,713 exact matches
```

The F28 computation is deterministic for a fixed input dataset, extraction rule, serialization implementation, and tested numerical range.

8. Interpretation & Limitations

Interpretation. This demonstrates a substantial and reproducible byte-level compatibility between the observed push[1] payload population and the tested historical CBigNum serialization model.

Limitations & Scope. The F28 finding is strictly a serialization-matching result representing 8,713 exact byte-level correspondences. It does not attempt to assign external identities, cryptographic attribution, or miner signatures to the observed transactions. Attribution is kept separate from the underlying serialization experiment.

9. Appendix — Self-Contained Reproduction Code

The following code reproduces the CBigNum-style serialization and observed-payload extraction used by the F28 computation. The input-loading function is explicitly defined so the appendix is self-contained with respect to the CSV input.

```
import pandas as pd

def bn_to_vch(n):
    if n == 0:
        return b''

    neg = n < 0
    abs_n = abs(n)

    vch = []
    while abs_n > 0:
        vch.append(abs_n & 0xff)
        abs_n >>= 8

    if vch[-1] & 0x80:
        vch.append(0x80 if neg else 0x00)
    elif neg:
        vch[-1] |= 0x80

    return bytes(vch)

def parse_push1_raw(hex_str):
    if pd.isna(hex_str) or not isinstance(hex_str, str):
        return None

    i = 0
    length = len(hex_str)
    pushes = []

    while i < length:
        if i + 2 > length:
            break

        opcode = int(hex_str[i:i+2], 16)
        i += 2

        if 1 <= opcode <= 75:
            push_len = opcode
        elif opcode == 76:
            if i + 2 > length:
                break
            push_len = int(hex_str[i:i+2], 16)
            i += 2
        elif opcode == 77:
            if i + 4 > length:
                break
            push_len = int(hex_str[i:i+4], 16)
            i += 4
        else:
            break

        data_end = i + (push_len * 2)
        if data_end > length:
            break

        pushes.append(hex_str[i:data_end])
        i = data_end

    return pushes[1].lower() if len(pushes) > 1 else None

def load_observed_hex_payloads(csv_path):
    df = pd.read_csv(csv_path)
    payloads = set()

    for hex_str in df["script_hex"].dropna():
        p1 = parse_push1_raw(hex_str)
        if p1:
            payloads.add(p1)

    return payloads

observed_push1_set = load_observed_hex_payloads(
    "scriptSig_analysis.csv"
)

matches = 0

for n in range(1, 200000):
```

```
serialized = bn_to_vch(n).hex().lower()

if serialized in observed_push1_set:
    matches += 1

print(f"Total F28 Exact Matches: {matches}")
```

Archived reference result: F28 Exact Match Count = **8,713** under the documented input dataset, extraction rule, serialization implementation, and numerical range.

Document status: Archived Reference Copy — Version 3 (Final)